Design Patterns: Complete Guide

With Dart/Flutter and Python Examples

Software Architecture Documentation

May 5, 2026

Contents

1 Introduction to Design Patterns

Definition: Design Pattern

A design pattern is a general, reusable solution to a commonly occurring problem within a given context in software design. It is not a finished design that can be transformed directly into code, but rather a template or description for how to solve a problem that can be used in many different situations.

1.1 Why Are Design Patterns Important?

Why Use This Pattern?

- **Reusability:** Patterns capture proven solutions, saving time and effort.
- **Communication:** Developers can say "use Singleton here" instead of lengthy explanations.
- **Best Practices:** Patterns embody decades of software engineering experience.
- **Maintainability:** Code becomes more modular, flexible, and easier to understand.
- **Avoid Reinventing:** You don't need to solve problems that others have already solved.
- **Documentation:** Patterns serve as high-level documentation of your architecture.

1.2 History of Design Patterns

The concept was popularized by the "Gang of Four" (GoF) - Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides - in their 1994 book "Design Patterns: Elements of Reusable Object-Oriented Software". They documented 23 patterns that remain relevant today.

1.3 Classification of Patterns

The Three Categories

- **Creational** (5 patterns)
  - Singleton
  - Factory Method
  - Abstract Factory
  - Builder
  - Prototype
- **Structural** (7 patterns)
  - Adapter
  - Decorator
  - Facade
  - Proxy
  - Bridge
  - Composite
  - Flyweight
- **Behavioral** (11 patterns)
  - Observer
  - Strategy
  - Command
  - Template Method
  - State
  - Chain of Responsibility
  - Iterator
  - Mediator
  - Memento
  - Visitor
  - Interpreter

2 Creational Patterns

Creational patterns deal with object creation mechanisms, trying to create objects in a manner suitable to the situation. They reduce complexities and instability by controlling the creation process.

2.1 Singleton Pattern

Definition: Singleton

Ensures a class has **only one instance** and provides a **global point of access** to that instance.

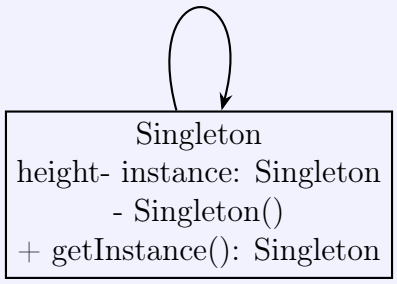
Why Use This Pattern?

- **When to use:** Database connections, logging services, configuration managers, thread pools, cache managers.
- **Problem it solves:** Multiple instances would cause inconsistencies (e.g., two configuration objects with different settings).
- **Benefits:** Controlled access to sole instance, reduced memory footprint, lazy initialization possible.

Real-World Analogy

Think of a country's government. There is only **one** official government for a country at any time. Everyone accesses the same government (global point of access). Creating another "fake government" would cause chaos!

UML Diagram



Only one instance exists

Code Example: Dart/Flutter

```
class DatabaseService {
  // Static variable holds the single instance
  static final DatabaseService _instance = DatabaseService.
   _internal();

```

```
  // Private constructor - prevents external instantiation
  DatabaseService._internal() {
    print("DatabaseService instance created!");
  }

  // Factory constructor returns the same instance
  factory DatabaseService() {
    return _instance;
  }

  void query(String sql) {
    print("Executing query: $sql");
  }
}

// Usage in Flutter
void main() {
  final db1 = DatabaseService();
  final db2 = DatabaseService();

  // Check if both references point to same object
  print(identical(db1, db2)); // true
  db1.query("SELECT * FROM users");
}
```

Listing 1: Singleton Pattern in Dart

Code Explanation

How it works:

- `_instance` is static → belongs to class, not objects
- `_internal()` is private → cannot call from outside
- `factory` constructor returns cached instance instead of creating new
- Every call to `DatabaseService()` returns same object

Code Example: Python

```python
class DatabaseConnection:
    _instance = None

    def __new__(cls):
        """Override __new__ to control instance creation"""
        if cls._instance is None:
            print("Creating new instance")
            cls._instance = super().__new__(cls)
            cls._instance.connection = None
        return cls._instance

    def connect(self):
        if not self.connection:
            self.connection = "Connected to DB"
        return self.connection
```

```

# Usage
db1 = DatabaseConnection()
db2 = DatabaseConnection()
print(db1 is db2)  # True
db1.connect()
print(db2.connection)  # "Connected to DB"
```

Listing 2: Singleton Pattern in Python

Pro Tip

In Flutter apps: Use `GetIt` package or `Provider` with `Singleton` for better testability instead of raw Singleton pattern.

Warning

Anti-pattern alert: Overusing Singleton can lead to:

- Hidden dependencies (code becomes hard to test)
- Global state problems (unexpected side effects)
- Concurrency issues in multi-threaded environments
- Difficulty in unit testing (can't easily mock)

2.2 Factory Method Pattern

Definition: Factory Method

Defines an interface for creating an object, but lets **subclasses decide which class to instantiate**. It defers instantiation to subclasses.

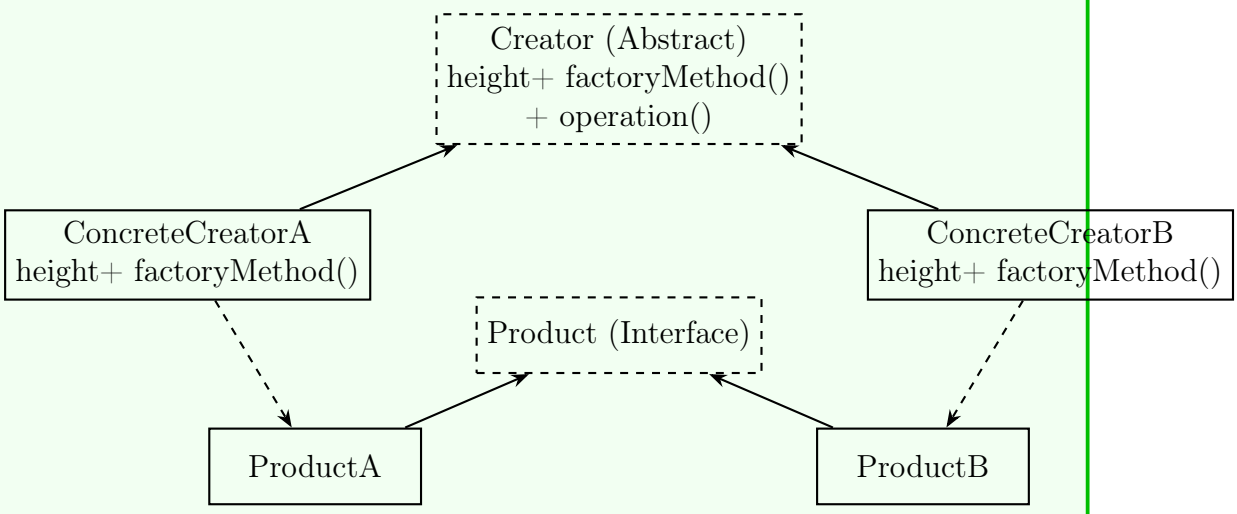
Why Use This Pattern?

- **When to use:** Class can't anticipate the type of objects it needs to create; subclasses specify which objects to create.
- **Problem it solves:** Direct instantiation (`new` operator) couples code to concrete classes.
- **Benefits:** Promotes loose coupling, follows Open/Closed principle, improves testability.

Real-World Analogy

Logistics Company: A shipping company doesn't know if a package will go by Truck, Ship, or Plane. They have a `createTransport()` method that different warehouses override based on package destination. The main logistics code works with any transport!

UML Diagram



Code Example: Python

```python
from abc import ABC, abstractmethod

# Product interface
class Button(ABC):
    @abstractmethod
    def render(self) -> str:
        pass

```

```
    @abstractmethod
    def on_click(self) -> None:
        pass

# Concrete Products
class WindowsButton(Button):
    def render(self) -> str:
        return "Rendering Windows-style button"

    def on_click(self) -> None:
        print("Windows button clicked!")

class MacButton(Button):
    def render(self) -> str:
        return "Rendering Mac-style button"

    def on_click(self) -> None:
        print("Mac button clicked with smooth animation!")

class LinuxButton(Button):
    def render(self) -> str:
        return "Rendering Linux/GTK button"

    def on_click(self) -> None:
        print("Linux button clicked!")

# Creator (Factory) abstract class
class Dialog(ABC):
    @abstractmethod
    def create_button(self) -> Button:
        """Factory method"""
        pass

    def render_window(self) -> None:
        """Common operation that uses the factory method"""
        button = self.create_button()
        result = button.render()
        print(result)
        button.on_click()

# Concrete Creators
class WindowsDialog(Dialog):
    def create_button(self) -> Button:
        return WindowsButton()

class MacDialog(Dialog):
    def create_button(self) -> Button:
        return MacButton()

class LinuxDialog(Dialog):
    def create_button(self) -> Button:
        return LinuxButton()

# Usage - Client code doesn't know which button it's getting!
def client_code(dialog: Dialog):
    dialog.render_window()

# Application decides which dialog based on OS
```

```
os_type = "Windows"  # This could come from config
if os_type == "Windows":
    dialog = WindowsDialog()
elif os_type == "Mac":
    dialog = MacDialog()
else:
    dialog = LinuxDialog()

client_code(dialog)
```

Listing 3: Factory Method in Python

Code Explanation

Key Points:

- `Dialog` is abstract - defines the factory method `create_button()`
- Each concrete dialog returns its specific button type
- `render_window()` works with ANY button - it's decoupled!
- Adding a new button type (e.g., `IOSButton`) only requires new subclass - no existing code changes!

Code Example: Dart/Flutter (Realistic UI Example)

```
import 'package:flutter/material.dart';

// Product interface
abstract class ThemeButton {
  Widget build();
  String get description;
}

// Concrete products
class LightThemeButton implements ThemeButton {
  @override
  Widget build() {
    return ElevatedButton(
      style: ElevatedButton.styleFrom(backgroundColor: Colors.white
   ),
      onPressed: () {},
      child: Text('Light Button', style: TextStyle(color: Colors.
   black)),
    );
  }

  @override
  String get description => "Light theme button";
}

class DarkThemeButton implements ThemeButton {
  @override
  Widget build() {
    return ElevatedButton(
```

```
      style: ElevatedButton.styleFrom(backgroundColor: Colors.grey[900]),
      onPressed: () {},
      child: Text('Dark Button', style: TextStyle(color: Colors.white)),
    );
  }

  @override
  String get description => "Dark theme button";
}

// Creator abstract class
abstract class ThemeFactory {
  ThemeButton createButton();

  Widget buildThemeSensitiveUI() {
    final button = createButton();
    return Column(
      children: [
        button.build(),
        Text(button.description),
      ],
    );
  }
}

// Concrete creators
class LightThemeFactory extends ThemeFactory {
  @override
  ThemeButton createButton() => LightThemeButton();
}

class DarkThemeFactory extends ThemeFactory {
  @override
  ThemeButton createButton() => DarkThemeButton();
}

// Flutter widget using the factory
class ThemeAwareScreen extends StatelessWidget {
  final bool isDarkMode;

  const ThemeAwareScreen({required this.isDarkMode});

  @override
  Widget build(BuildContext context) {
    final factory = isDarkMode
        ? DarkThemeFactory()
        : LightThemeFactory();

    return Scaffold(
      appBar: AppBar(title: Text('Factory Method Demo')),
      body: Center(
        child: factory.buildThemeSensitiveUI(),
      ),
    );
  }
}
```

Listing 4: Factory Method in Flutter - Theme-Aware Widgets

Pro Tip

When to prefer Factory Method over direct instantiation:

- You don't know ahead of time what class you need
- Your class needs to be extended with new product types
- You want to centralize object creation for consistency

3 Structural Patterns

Structural patterns explain how to assemble objects and classes into larger structures while keeping them flexible and efficient.

3.1 Adapter Pattern

Definition: Adapter

Converts the interface of a class into another interface that clients expect. It allows classes to work together that couldn't otherwise because of incompatible interfaces.

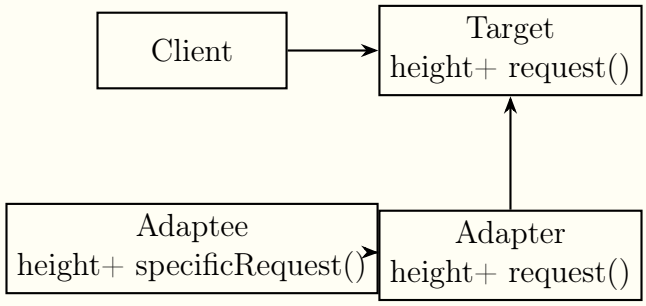
Why Use This Pattern?

- **When to use:** Integrating legacy code, using third-party libraries with different interfaces, making incompatible classes work together.
- **Problem it solves:** You have a class that does what you need, but its interface doesn't match your system.
- **Benefits:** Reusability of existing classes, no need to modify existing code, Single Responsibility Principle (interface conversion is separated).

Real-World Analogy

Power Adapter: When you travel from US to Europe, your laptop charger has a US plug (interface), but European sockets are different. You use a travel adapter (Adapter pattern) that converts the European socket interface to the US plug interface your charger expects.

UML Diagram



Code Example: Python (Payment Gateway Integration)

```python
from abc import ABC, abstractmethod

# Target interface (what our system expects)
class PaymentProcessor(ABC):
    @abstractmethod
    def process_payment(self, amount: float, currency: str) -> bool
    :
```

```
        pass

    @abstractmethod
    def refund_payment(self, transaction_id: str) -> bool:
        pass

# Adaptee 1 - Stripe API (incompatible interface)
class StripeAPI:
    def create_charge(self, amount_cents: int, currency_code: str)
    -> dict:
        print(f"Stripe: Charging {amount_cents} {currency_code}")
        return {"success": True, "txn_id": "stripe_123"}

    def reverse_charge(self, charge_id: str) -> dict:
        print(f"Stripe: Refunding {charge_id}")
        return {"success": True}

# Adaptee 2 - PayPal API (different interface)
class PayPalAPI:
    def make_payment(self, amount_usd: float) -> str:
        print(f"PayPal: Processing ${amount_usd}")
        return "paypal_txn_456"

    def cancel_payment(self, transaction_token: str) -> bool:
        print(f"PayPal: Cancelling {transaction_token}")
        return True

# Adapter for Stripe
class StripeAdapter(PaymentProcessor):
    def __init__(self):
        self.stripe = StripeAPI()

    def process_payment(self, amount: float, currency: str) -> bool
    :
        # Convert our interface to Stripe's expected format
        amount_cents = int(amount * 100)
        response = self.stripe.create_charge(amount_cents, currency
    )
        return response.get("success", False)

    def refund_payment(self, transaction_id: str) -> bool:
        response = self.stripe.reverse_charge(transaction_id)
        return response.get("success", False)

# Adapter for PayPal
class PayPalAdapter(PaymentProcessor):
    def __init__(self):
        self.paypal = PayPalAPI()
        self.transactions = {}  # Store mapping

    def process_payment(self, amount: float, currency: str) -> bool
    :
        # Convert currency to USD (simplified)
        if currency != "USD":
            amount = amount * 1.1  # Fake conversion
        txn_id = self.paypal.make_payment(amount)
        self.transactions[txn_id] = True
        return True
```

```python

    def refund_payment(self, transaction_id: str) -> bool:
        return self.paypal.cancel_payment(transaction_id)

# Client code - works with any payment processor!
class ShoppingCart:
    def __init__(self, processor: PaymentProcessor):
        self.processor = processor

    def checkout(self, amount: float):
        print(f"Processing checkout of ${amount}")
        success = self.processor.process_payment(amount, "USD")
        if success:
            print("Payment successful!")
        else:
            print("Payment failed!")

# Usage
cart = ShoppingCart(StripeAdapter())
cart.checkout(49.99)

cart2 = ShoppingCart(PayPalAdapter())
cart2.checkout(29.99)
```

Listing 5: Adapter Pattern for Payment Systems

Code Explanation

What's happening:

- Our system expects `process_payment(amount, currency)`
- Stripe expects `create_charge(amount_cents, currency_code)`
- PayPal expects `make_payment(amount_usd)`
- Adapters **translate** between our interface and their interface
- ShoppingCart works with ANY processor through the same interface!

Code Example: Dart/Flutter (API Response Adapter)

```dart
// Existing third-party API response (can't change)
class OldApiResponse {
  final Map<String, dynamic> data;

  OldApiResponse(this.data);

  String get userName => data['user_name'];
  int get userAge => data['user_age'];
}

// Our app's expected model
class User {
  final String name;
  final int age;
```

```
15
16    User({required this.name, required this.age});
17  }
18
19  // Target interface
20  abstract class UserDataSource {
21    Future<User> getUser(String id);
22  }
23
24  // Adapter that makes old API work with our interface
25  class OldApiAdapter implements UserDataSource {
26    final OldApiResponse Function(String) fetchFromOldApi;
27
28    OldApiAdapter(this.fetchFromOldApi);
29
30    @override
31    Future<User> getUser(String id) async {
32      // Call old API
33      final response = fetchFromOldApi(id);
34
35      // Adapt the response to our expected format
36      return User(
37        name: response.userName,
38        age: response.userAge,
39      );
40    }
41  }
42
43  // New modern API that already matches our interface
44  class NewApi implements UserDataSource {
45    @override
46    Future<User> getUser(String id) async {
47      // Direct implementation
48      return User(name: "John", age: 30);
49    }
50  }
51
52  // Our app's service (works with any UserDataSource)
53  class UserService {
54    final UserDataSource dataSource;
55
56    UserService(this.dataSource);
57
58    Future<void> displayUser(String id) async {
59      final user = await dataSource.getUser(id);
60      print("User: ${user.name}, Age: ${user.age}");
61    }
62  }
63
64  void main() {
65    // Without adapter - using old API
66    final oldApi = OldApiAdapter((id) => OldApiResponse({
67      'user_name': 'Alice',
68      'user_age': 25
69    }));
70    final service = UserService(oldApi);
71    service.displayUser("123");
72
```

```
  // With modern API - no adapter needed!
  final modernService = UserService(NewApi());
  modernService.displayUser("456");
}
```

Listing 6: Adapter Pattern in Flutter for API Data

Pro Tip

Adapter vs. Facade:

- **Adapter**: Converts one interface to another (usually to make existing code work)
- **Facade**: Provides a simplified interface to a complex subsystem (hides complexity)
- Both wrap existing classes, but with different intent!

4 Behavioral Patterns

Behavioral patterns are concerned with algorithms and the assignment of responsibilities between objects.

4.1 Observer Pattern

Definition: Observer

Defines a one-to-many dependency between objects so that when one object (the Subject) changes state, all its dependents (Observers) are notified and updated automatically.

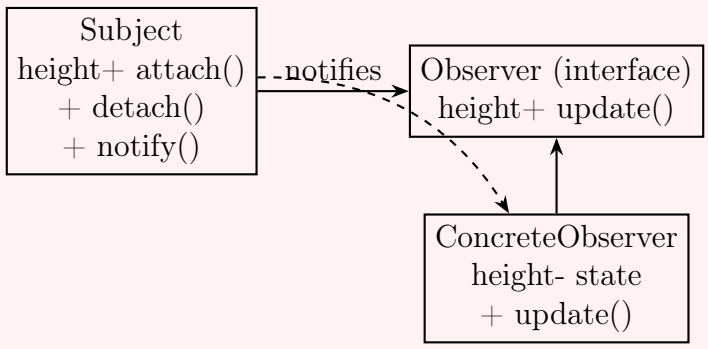
Why Use This Pattern?

- **When to use:** Event handling systems, real-time data feeds, UI updates, model-view-controller (MVC) architectures.
- **Problem it solves:** You need many objects to stay in sync with another object's state without tight coupling.
- **Benefits:** Loose coupling (subject only knows observers implement an interface), broadcast communication, dynamic relationships (observers can be added/removed at runtime).

Real-World Analogy

YouTube Subscriptions: When a YouTuber (Subject) publishes a new video, all subscribers (Observers) get a notification. Subscribers can subscribe/unsubscribe anytime. The YouTuber doesn't need to know each subscriber individually - just that they want notifications!

UML Diagram



Code Example: Dart/Flutter (Using Streams)

```
import 'dart:async';

// Subject (Observable)
class NewsAgency {
```

```
  // StreamController manages the observers
  final _controller = StreamController<String>();

  // Getter for the stream (observers subscribe to this)
  Stream<String> get newsStream => _controller.stream;

  // Method to publish news (notify all observers)
  void publishNews(String news) {
    print("News Agency: Publishing '$news'");
    _controller.add(news);  // This notifies all subscribers
  }

  void dispose() {
    _controller.close();
  }
}

// Observer 1 - Mobile App Notification
class MobileNotification {
  final String deviceId;

  MobileNotification(this.deviceId);

  void update(String news) {
    print("       Mobile ($deviceId): Breaking News - $news");
  }
}

// Observer 2 - Email Alert System
class EmailAlert {
  final String email;

  EmailAlert(this.email);

  void update(String news) {
    print("       Email to $email: $news");
  }
}

// Observer 3 - Desktop Widget
class DesktopWidget {
  void update(String news) {
    print("          Desktop Widget: Latest news - $news");
  }
}

void main() {
  final agency = NewsAgency();
  final mobileNews = MobileNotification("iPhone12");
  final emailAlert = EmailAlert("user@example.com");
  final desktopWidget = DesktopWidget();

  // Subscribe observers to the stream
  agency.newsStream.listen((news) => mobileNews.update(news));
  agency.newsStream.listen((news) => emailAlert.update(news));
  agency.newsStream.listen((news) => desktopWidget.update(news));

  // Publish news - all observers get notified!
```

```
  agency.publishNews("Flutter 3.16 Released!");
  agency.publishNews("Dart 3.2 is out!");

  agency.dispose();
}
```

Listing 7: Observer Pattern in Flutter with Streams

Code Explanation

How Dart Streams implement Observer:

- `StreamController` is the Subject
- `stream.listen(observer)` attaches observers
- `controller.add(data)` notifies all observers automatically
- Perfect for Flutter's reactive UI paradigm!

Code Example: Python (Manual Implementation)

```python
from abc import ABC, abstractmethod
from typing import List

# Observer interface
class Observer(ABC):
    @abstractmethod
    def update(self, temperature: float, humidity: float) -> None:
        pass

# Subject (Observable)
class WeatherStation:
    def __init__(self):
        self._observers: List[Observer] = []
        self._temperature = 0.0
        self._humidity = 0.0

    def attach(self, observer: Observer) -> None:
        """Add an observer"""
        if observer not in self._observers:
            self._observers.append(observer)
            print(f"Attached observer: {observer.__class__.__name__}")

    def detach(self, observer: Observer) -> None:
        """Remove an observer"""
        self._observers.remove(observer)
        print(f"Detached observer: {observer.__class__.__name__}")

    def notify(self) -> None:
        """Notify all observers of state change"""
        for observer in self._observers:
            observer.update(self._temperature, self._humidity)

```

```python
    def set_measurements(self, temp: float, humidity: float) -> None:
        """Change state and notify observers"""
        self._temperature = temp
        self._humidity = humidity
        print(f"\nWeather Station: New measurements - Temp: {temp}C , Humidity: {humidity}%")
        self.notify()

# Concrete Observer 1
class PhoneDisplay(Observer):
    def __init__(self, device_name: str):
        self.device_name = device_name

    def update(self, temperature: float, humidity: float) -> None:
        print(f"       {self.device_name} Display: {temperature} C, {humidity}% humidity")

# Concrete Observer 2
class WebsiteDisplay(Observer):
    def update(self, temperature: float, humidity: float) -> None:
        print(f"       Website Updated: Current temperature is {temperature} C ")

# Concrete Observer 3
class EmergencyAlert(Observer):
    def update(self, temperature: float, humidity: float) -> None:
        if temperature > 35:
            print(f"       HEAT ALERT! Temperature {temperature} C exceeds safe limit!")

# Usage
station = WeatherStation()

# Create observers
phone1 = PhoneDisplay("iPhone")
phone2 = PhoneDisplay("Samsung")
website = WebsiteDisplay()
alert = EmergencyAlert()

# Attach observers
station.attach(phone1)
station.attach(phone2)
station.attach(website)
station.attach(alert)

# Change state - all observers get notified
station.set_measurements(25.5, 60)
station.set_measurements(38.0, 45)  # This triggers emergency alert!

# Remove an observer
station.detach(phone2)
station.set_measurements(22.0, 55)  # Samsung phone won't get this update
```

Listing 8: Observer Pattern in Python

Code Explanation

Observer Pattern in Python:

- Subject maintains list of observers
- Observers implement `update()` method
- Subject calls `update()` on all observers when state changes
- Observers can be added/removed at runtime

Pro Tip

In Flutter Development: Flutter's entire UI framework is built on Observer pattern! `StatefulWidget` and `setState()` notify Flutter to rebuild. `Provider`, `Bloc`, and `GetX` are all implementations of Observer pattern.

4.2 Strategy Pattern

Definition: Strategy

Defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

Why Use This Pattern?

- **When to use:** Multiple ways to perform an operation; you want to avoid massive if-else or switch statements.
- **Problem it solves:** Classes often have many variants of an algorithm, leading to code duplication.
- **Benefits:** Open/Closed principle (add new strategies without changing context), eliminates conditional statements, improves testability.

Real-World Analogy

Navigation App: Google Maps has different routing strategies: "Driving", "Walking", "Biking", "Public Transit". The app (Context) doesn't care which strategy you choose - it just calls `calculateRoute()`. Each strategy provides its own implementation!

Code Example: Python (Payment Strategies)

```python
from abc import ABC, abstractmethod

# Strategy interface
class PaymentStrategy(ABC):
    @abstractmethod
    def pay(self, amount: float) -> None:
        pass

# Concrete Strategy 1: Credit Card
class CreditCardPayment(PaymentStrategy):
    def __init__(self, card_number: str, cvv: str):
        self.card_number = card_number
        self.cvv = cvv

    def pay(self, amount: float) -> None:
        print(f"       Paid ${amount} using Credit Card ending in {
    self.card_number[-4:]}")

# Concrete Strategy 2: PayPal
class PayPalPayment(PaymentStrategy):
    def __init__(self, email: str):
        self.email = email

    def pay(self, amount: float) -> None:
        print(f"       Paid ${amount} using PayPal account {self.
    email}")
```

```python

# Concrete Strategy 3: Cryptocurrency
class CryptoPayment(PaymentStrategy):
    def __init__(self, wallet_address: str):
        self.wallet_address = wallet_address

    def pay(self, amount: float) -> None:
        # Convert to BTC (simplified)
        btc_amount = amount / 50000
        print(f"    Paid {btc_amount:.6f} BTC (${amount}) from wallet {self.wallet_address[:6]}...")

# Concrete Strategy 4: Buy Now Pay Later
class BNPLPayment(PaymentStrategy):
    def pay(self, amount: float) -> None:
        installments = amount / 4
        print(f"       Paid ${amount} in 4 installments of ${installments:.2f}")

# Context - Shopping Cart
class ShoppingCart:
    def __init__(self):
        self.items = []
        self.payment_strategy: PaymentStrategy = None

    def add_item(self, item_name: str, price: float):
        self.items.append({"name": item_name, "price": price})
        print(f"Added {item_name} - ${price}")

    def set_payment_strategy(self, strategy: PaymentStrategy):
        self.payment_strategy = strategy

    def calculate_total(self) -> float:
        return sum(item["price"] for item in self.items)

    def checkout(self):
        total = self.calculate_total()
        print(f"\nTotal: ${total}")

        if not self.payment_strategy:
            raise Exception("Please select a payment method!")

        self.payment_strategy.pay(total)
        print("Order completed!       \n")

# Usage
cart = ShoppingCart()
cart.add_item("Laptop", 1200)
cart.add_item("Mouse", 25)

# Pay with Credit Card
cart.set_payment_strategy(CreditCardPayment("1234-5678-9012-3456", "123"))
cart.checkout()

# Pay with PayPal
cart.set_payment_strategy(PayPalPayment("user@example.com"))
cart.checkout()
```

```
80
81 # Pay with Crypto
82 cart.set_payment_strategy(CryptoPayment("1
       A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa"))
83 cart.checkout()
```

Listing 9: Strategy Pattern for Payment Methods

Code Explanation

Strategy Pattern Benefits:

- Payment methods are completely independent
- Easy to add new payment method (just implement `PaymentStrategy`)
- ShoppingCart doesn't need to change when we add new payment types
- Client code chooses which strategy to use at runtime

Code Example: Dart/Flutter (Image Filter Strategies)

```
1  import 'dart:typed_data';
2  import 'dart:ui' as ui;
3
4  // Strategy interface
5  abstract class ImageFilterStrategy {
6    String get name;
7    Future<ui.Image> apply(ui.Image image);
8  }
9
10 // Concrete Strategy 1: Grayscale Filter
11 class GrayscaleFilter implements ImageFilterStrategy {
12   @override
13   String get name => 'Grayscale';
14
15   @override
16   Future<ui.Image> apply(ui.Image image) async {
17     print("Applying grayscale filter...");
18     // In real implementation, you'd manipulate pixels
19     return image; // Simplified for demo
20   }
21 }
22
23 // Concrete Strategy 2: Sepia Filter
24 class SepiaFilter implements ImageFilterStrategy {
25   @override
26   String get name => 'Sepia';
27
28   @override
29   Future<ui.Image> apply(ui.Image image) async {
30     print("Applying sepia filter for vintage look...");
31     return image;
32   }
33 }
34
```

```
// Concrete Strategy 3: Blur Filter
class BlurFilter implements ImageFilterStrategy {
  final double sigma;

  BlurFilter({this.sigma = 5.0});

  @override
  String get name => 'Blur (${sigma}px)';

  @override
  Future<ui.Image> apply(ui.Image image) async {
    print("Applying blur with sigma ${sigma}...");
    return image;
  }
}

// Strategy Context - Image Editor
class ImageEditor {
  ImageFilterStrategy? _currentFilter;

  void setFilter(ImageFilterStrategy filter) {
    _currentFilter = filter;
    print("Filter changed to: ${filter.name}");
  }

  Future<void> processImage(ui.Image image) async {
    if (_currentFilter == null) {
      print("No filter selected, using original image");
      return;
    }

    print("Processing image with ${_currentFilter!.name} filter");
    final processed = await _currentFilter!.apply(image);
    print("Image processing complete!");
  }
}

// Flutter Widget Example
class PhotoEditorScreen extends StatefulWidget {
  @override
  _PhotoEditorScreenState createState() => _PhotoEditorScreenState
    ();
}

class _PhotoEditorScreenState extends State<PhotoEditorScreen> {
  final editor = ImageEditor();
  ImageFilterStrategy? selectedFilter;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Photo Editor')),
      body: Column(
        children: [
          // Image preview would go here
          SizedBox(height: 20),
          Text('Select a filter:'),
          Row(
```

```
            mainAxisAlignment: MainAxisAlignment.spaceEvenly,
            children: [
              ElevatedButton(
                onPressed: () {
                  setState(() {
                    selectedFilter = GrayscaleFilter();
                    editor.setFilter(selectedFilter!);
                  });
                },
                child: Text('Grayscale'),
              ),
              ElevatedButton(
                onPressed: () {
                  setState(() {
                    selectedFilter = SepiaFilter();
                    editor.setFilter(selectedFilter!);
                  });
                },
                child: Text('Sepia'),
              ),
              ElevatedButton(
                onPressed: () {
                  setState(() {
                    selectedFilter = BlurFilter(sigma: 8.0);
                    editor.setFilter(selectedFilter!);
                  });
                },
                child: Text('Blur'),
              ),
            ],
          ),
          if (selectedFilter != null)
            Padding(
              padding: EdgeInsets.all(16.0),
              child: Text('Active: ${selectedFilter!.name}'),
            ),
        ],
      ),
    );
  }
}
```

Listing 10: Strategy Pattern in Flutter for Image Filters

Pro Tip

Strategy vs. State Pattern:

- **Strategy**: Client decides which algorithm to use (behaviors are independent)
- **State**: Object's behavior changes based on its internal state (states know about each other)
- Strategy is a way to configure a class with one of many behaviors

5 Practical Comparison Table

Pattern Decision Matrix

Pattern	Use When	Avoid When	Flutter Example
Singleton	Exactly one instance needed globally	Testing is critical (hard to mock)	DatabaseService, ApiClient
Factory Method	Class can't know exact type to create	Simple creation (new is fine)	Theme-aware widgets
Adapter	Incompatible interfaces need to work	You control all code (just change it)	API response format conversion
Observer	One-to-many notification needed	Simple direct calls work fine	Provider, StreamBuilder, BLoC
Strategy	Multiple algorithms interchangeable	Only 1-2 variants (simple if-else is fine)	Payment methods, validators
Decorator	Need to add responsibilities dynamically	Complex hierarchy develops	ScrollBehavior, CustomPaint
Facade	Simplify complex subsystem	Subsystem is already simple	Flutter's Navigator (simplifies routing)

6 Common Anti-Patterns to Avoid

Warning

Pattern Misuse Examples:

1. **Singleton Overuse**: Creating singletons for everything leads to global state spaghetti code.
2. **Golden Hammer**: Using design patterns everywhere, even when simple code would work.
3. **Cargo Cult**: Implementing a pattern without understanding the problem it solves.
4. **Over-engineering**: Adding Factory + Strategy + Observer for a simple counter app.

7 Choosing the Right Pattern: Step-by-Step

Decision Process

1. **Identify what varies** - Find what changes in your system
2. **Encapsulate the variation** - Hide it behind an interface
3. **Consider the trade-offs** - Complexity vs. flexibility vs. performance
4. **Start simple** - Implement the simplest solution first
5. **Refactor to pattern** - Only add pattern when you need it

8 Practice Exercises

Exercises for Mastery

1. **Singleton**: Implement a thread-safe logger in Python/Dart
2. **Factory**: Create a document generator that produces PDF, Word, and HTML
3. **Adapter**: Integrate two different weather APIs with different response formats
4. **Observer**: Build a stock price notifier with multiple displays
5. **Strategy**: Create a sorting app that can switch between bubble, quick, and merge sort

9 Conclusion

Design patterns are powerful tools in every developer's toolkit. Remember these key principles:

- **Patterns are solutions to recurring problems**, not silver bullets
- **Learn the problem first**, then the pattern that solves it
- **Prefer composition over inheritance** - many patterns use composition
- **Program to interfaces, not implementations**
- **Favor delegation over inheritance** for flexibility
- **Don't be afraid to refactor** - patterns emerge naturally from clean code

"A pattern is a reusable solution to a recurring problem in a specific context."

In Flutter development specifically, you'll naturally use:

- **Observer** everywhere (Widget rebuilds, Provider, BLoC)
- **Builder** pattern (Widget builders, FutureBuilder, StreamBuilder)
- **Strategy** (Different animations, validators)
- **Adapter** (Platform channels for native code integration)

Keep this document as a reference, but remember: **the best pattern is the one that makes your code most readable and maintainable for your specific situation**.

Last updated: May 5, 2026